

Assignment 6 of MATP4820

Izaak King

March 20, 2026

Problem 1

Consider the constrained quadratic minimization problem:

$$\underset{\mathbf{x}}{\text{minimize}} f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}, \text{ s.t. } l_i \leq x_i \leq u_i, \forall i = 1, \dots, n, \quad (\text{C-QuadMin})$$

where $\mathbf{A} \in \mathbb{R}^{n \times n}$ is a symmetric positive definite matrix, $\mathbf{b} \in \mathbb{R}^n$, and $\mathbf{l} \in \mathbb{R}^n$ and $\mathbf{u} \in \mathbb{R}^n$ are lower and upper bounds.

1. Let $\mathbf{A} = \begin{bmatrix} 4 & -1 \\ -1 & 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, $l_1 = l_2 = 0$, and $u_1 = u_2 = 1$. Set the initial vector $\mathbf{x}^0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

- (a) Do by hand two iterations the projected gradient method with constant step size $\alpha_k = 0.25, \forall k$

First, note

$$\nabla f(x) = Ax - b$$

$$x^{(k+1)} = \text{Proj}_X (x^{(k)} - \alpha_k \nabla f(x^{(k)}))$$

$$\text{Proj}_{[a_i, b_i]}(x) = \min\{\max\{x, a_i\}, b_i\}$$

Now, iteration 1:

$$\nabla f(x^{(0)}) = Ax^{(0)} - b = \begin{bmatrix} 4 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \end{bmatrix}$$

$$x^{(0)} - \alpha_k \nabla f(x^{(0)}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.25 \begin{bmatrix} -2 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$

$$x^{(1)} = \text{Proj}_{[0,1]} \left(\begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} \right) = \min \left\{ \max \left\{ \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\} = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix}$$

Iteration 2:

$$\nabla f(x^{(1)}) = Ax^{(1)} - b = \begin{bmatrix} 4 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.25 \\ -1.25 \end{bmatrix}$$

$$x^{(1)} - \alpha_k \nabla f(x^{(1)}) = \begin{bmatrix} 0.5 \\ 0.25 \end{bmatrix} - 0.25 \begin{bmatrix} -0.25 \\ -1.25 \end{bmatrix} = \begin{bmatrix} 0.5625 \\ 0.5625 \end{bmatrix}$$

$$x^{(2)} = \text{Proj}_{[0,1]} \left(\begin{bmatrix} 0.5625 \\ 0.5625 \end{bmatrix} \right) = \min \left\{ \max \left\{ \begin{bmatrix} 0.5625 \\ 0.5625 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\} = \begin{bmatrix} 0.5625 \\ 0.5625 \end{bmatrix}$$

Thus, after two iterations of the projected gradient method, we get

$$\boxed{x^{(2)} = \begin{bmatrix} 0.5625 \\ 0.5625 \end{bmatrix}}$$

(b) Do by hand two cycles of the alternating minimization method.

First,

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x$$

$$\implies f(x) = \frac{1}{2} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^\top \begin{bmatrix} 4 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix}^\top \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\implies f(x) = \frac{1}{2}(4x_1^2 + x_2^2 - 2x_1x_2) - (2x_1 + x_2) = 2x_1^2 + \frac{1}{2}x_2^2 - x_1x_2 - 2x_1 - x_2$$

Now,

$$x_i^{(k+1)} = \arg \min_{x_i} f(x_1, x_2)$$

Setting derivatives equal to zero:

$$\text{For } x_1 : \frac{df}{dx_1} = 4x_1 - x_2 - 2 = \implies x_1 = \frac{x_2 + 2}{4}$$

$$\text{For } x_2 : \frac{df}{dx_2} = x_2 - x_1 - 1 \implies x_2 = x_1 + 1$$

Iteration 1:

$$x_1^{(1)} = \arg \min_{x_1} f(x_1, x_2^{(0)}) \implies x_1^{(1)} = \frac{0 + 2}{4} = \frac{1}{2} \in [0, 1]$$

$$x_2^{(1)} = \arg \min_{x_2} f(x_1^{(1)}, x_2) \implies x_2^{(1)} = \frac{1}{2} + 1 = \frac{3}{2} \notin [0, 1] \implies x_2^{(1)} = 1$$

Thus,

$$x^{(1)} = \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix}$$

Iteration 2:

$$x_1^{(2)} = \arg \min_{x_1} f(x_1, x_2^{(1)}) \implies x_1^{(2)} = \frac{1 + 2}{4} = \frac{3}{4} \in [0, 1]$$

$$x_2^{(2)} = \arg \min_{x_2} f(x_1^{(2)}, x_2) \implies x_2^{(2)} = \frac{3}{4} + 1 = \frac{7}{4} \notin [0, 1] \implies x_2^{(2)} = 1$$

Therefore, after two iterations of the alternating minimization method, we get:

$$x^{(2)} = \begin{bmatrix} 0.75 \\ 1 \end{bmatrix}$$

-
2. Use the instructor's provided file `quadMin_pg.m` to write a Matlab function `quadMin_pg` for solving (C-QuadMin) with input **A**, **b**, **l**, **u**, initial vector **x0**, and maximum iteration number **maxit**. Also test your function by running the provided test file `test_pg.m` and compare to the instructor's function. Print your code and the results you get.

Projected Gradient Method Function

```
function [x, hist_obj] = quadMin_pg(A,b,x0,maxit,lb,ub)

% projected gradient method for solving
% min_x 0.5*x'*A*x - b'*x
% s.t. lb <= x <= ub

x = x0;

% compute gradient of the objective
grad = A * x - b;

% compute the Lipschitz constant of grad
L = norm(A,2);

hist_obj = .5 * x' * A * x - b' * x;

for iter = 1:maxit

    % update x by projected gradient with step size 1/L
    x = min(max(x -(1/L) * grad, lb), ub);

    % compute gradient of the objective
    grad = A* x - b;

    % save objective value
    hist_obj = [hist_obj; .5*(x'* (grad - b))];

end

end
```

Code Results: `test_pg.m` Output

```
>> test_pg
```

```
Minimum found that satisfies the constraints.
```

```
Optimization completed because the objective function is non-decreasing in  
feasible directions, to within the value of the optimality tolerance,  
and constraints are satisfied to within the value of the constraint tolerance.
```

```
<stopping criteria details>
```

```
Student code: Total running time is 0.7985
```

```
Final objective value is -42.2517
```

```
Instructor code: Total running time is 0.7174
```

```
Final objective value is -42.2517
```

Student Figure Result

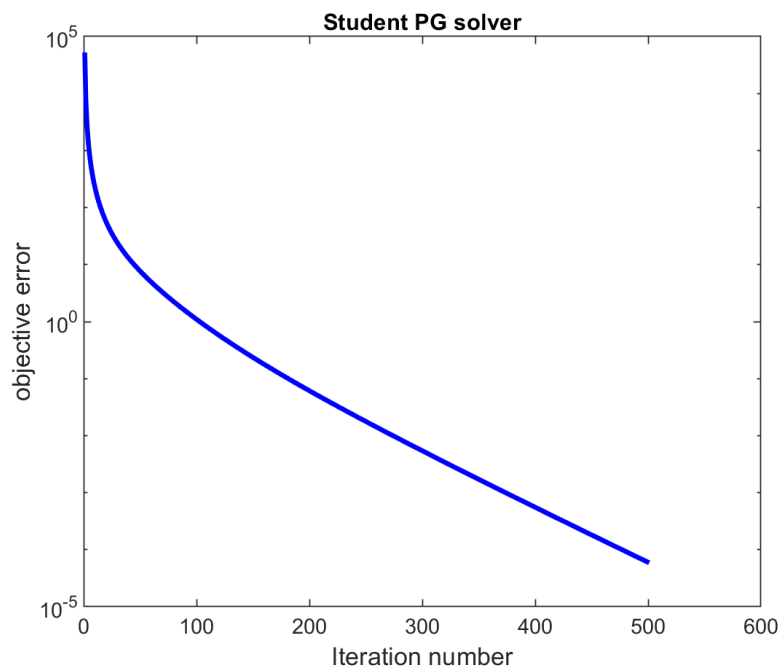


Figure 1: Student `test_pg` Result

Instructor Figure Result

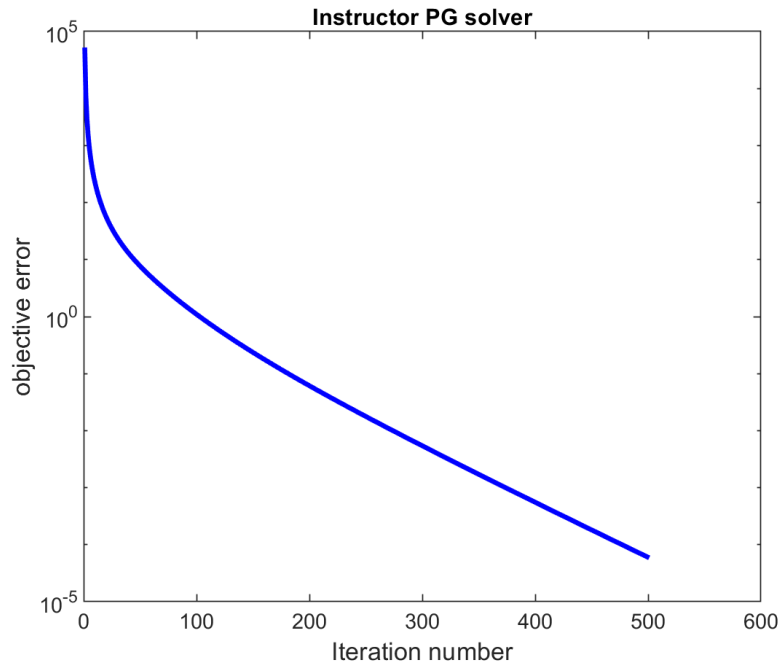


Figure 2: Instructor test_pg Result

Therefore, we have nearly identical performances of functions, so we conclude our quadMin_pg function is working correctly.

3. For any $i = 1, \dots, n$, derive the solution for the one-dimensional minimization problem

$$\min_{x_i} f(x), \text{ s.t. } l_i \leq x_i \leq u_i.$$

Note that the variable is x_i , while all $x_j, j \neq i$ are fixed. Your solution will depend on $x_j, j \neq i$ but not x_i .

First ,

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x$$

$$\nabla f(x) = Ax - b$$

Partial derivative with respect to x_i :

$$\frac{df}{dx_i} = (Ax)_i - b_i = \sum_{j=1}^n A_{ij}x_j - b_i$$

Separating x_i term

$$\frac{df}{dx_i} = A_{ii}x_i + \sum_{j \neq i} A_{ij}x_j - b_i$$

Now, as stated in the problem description, A is symmetric positive definite, so $A_{ii} > 0$, and thus any critical point will be a global minimum. Now setting derivative equal to zero to find minimizer x_i^* :

$$A_{ii}x_i^* + \sum_{j \neq i} A_{ij}x_j - b_i = 0 \implies x_i^* = \frac{b_i - \sum_{j \neq i} A_{ij}x_j}{A_{ii}}$$

Now applying constraints:

If $x_i^* < l_i$, the minimum is l_i .

If $x_i^* > u_i$, the minimum is u_i .

If $l_i < x_i^* < u_i$, the minimum is x_i^* .

Thus,

$$x_i^{(k+1)} = \max(l_i, \min(u_i, x_i^*))$$

$$x_i^{(k+1)} = \max\left(l_i, \min\left(u_i, \frac{b_i - \sum_{j \neq i} A_{ij} x_j^{(k)}}{A_{ii}}\right)\right)$$

-
4. Use the provided file `quadMin_AltMin.m` and the solution you derive in item 3 to write a Matlab function `quadMin_AltMin` of the alternating minimization method for solving (C-QuadMin). The input are **A**, **b**, **l**, **u**, initial vector **x0**, and maximum iteration number **maxit**. Also test your function by running the provided test file `test_AltMin.m` and compare to the instructor's `AltMin` function and PG (projected gradient) function. Print your code and the results you get. What do you observe on the convergence speed (in terms of number of cycles) of the `AltMin` and PG methods?

Alternating Minimization Method Function

```
function [x, hist_obj] = quadMin_AltMin(A,b,x0,maxit,lb,ub)

% alternating minimization method for solving
% min_x 0.5*x'*A*x - b'*x
% s.t. lb <= x <= ub

x = x0;

% compute the gradient and maintain it
r = A*x - b;

hist_obj = .5*(x'* (r - b));

n = length(b);

for iter = 1:maxit

    % update all coordinates cyclicly
    for i = 1:n

        % store old value

        x_old = x(i);

        % update x(i)

        x(i) = max(lb(i), min(ub(i), x(i) - r(i)/A(i,i))));
```

```
% update r vector in an efficient way

r = r + A(:,i) * (x(i) - x_old);
end

% save objective value after each cycle
hist_obj = [hist_obj; .5*(x'* (r - b))];

end

end
```

Code Results: test_pg.m Output

```
>> test_AltMin
```

Minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in feasible directions, to within the value of the optimality tolerance, and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>

Student code: Total running time is 0.8972

Final objective value is -44.5805

Instructor code: Total running time is 0.8992

Final objective value is -44.5805

Instructor code: Total running time is 0.6958

Final objective value is -44.5726

Student Figure Result

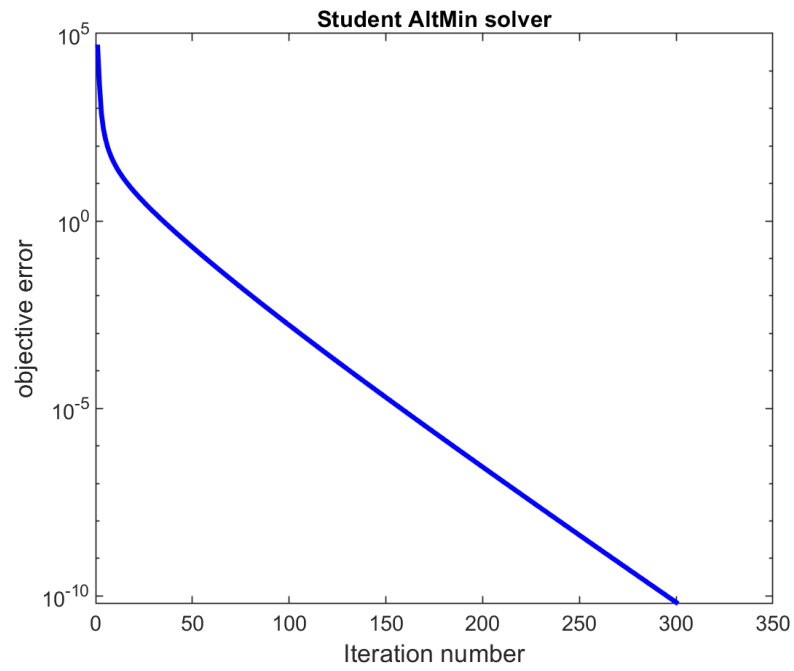


Figure 3: Student test_AltMin Result

Instructor Figure Result (AltMin)

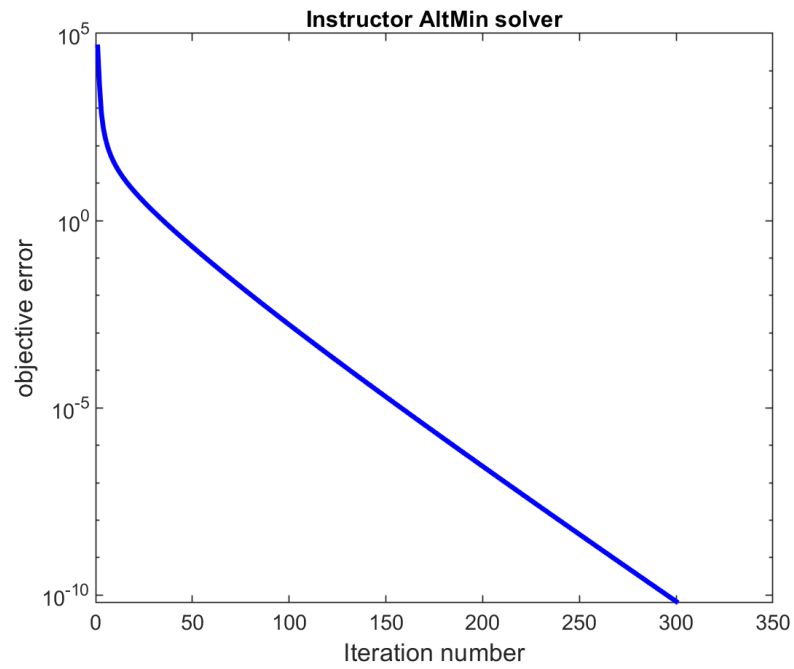


Figure 4: Instructor test_AltMin (AltMin) Result

Instructor Figure Result (PG)

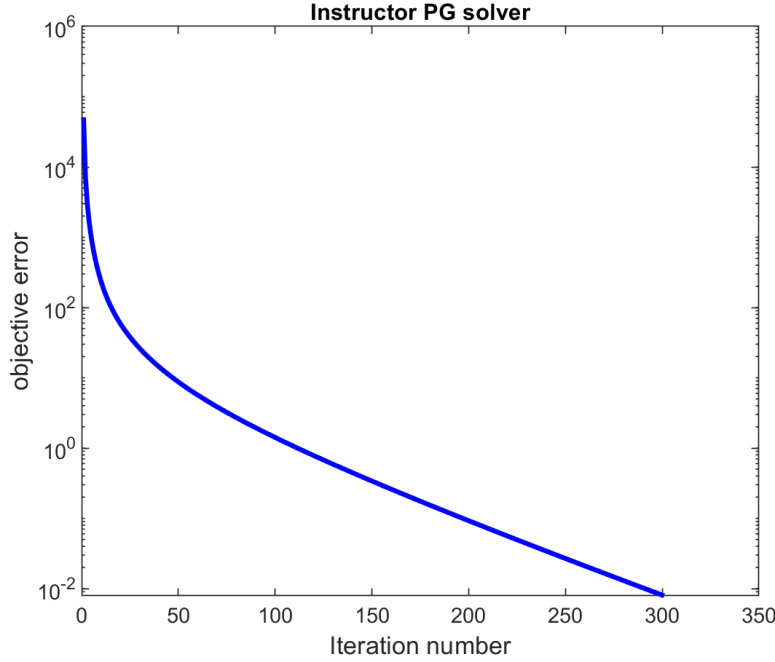


Figure 5: Instructor test_AltMin (PG) Result

We have nearly identical performances of AltMin functions, so we conclude our quad-Min_AltMin function is working correctly.

Now we will compare the results of the AltMin and PG methods. Based on the outputs, although the PG method has faster running time, the alternating minimization (AltMin) method makes more progress per cycle due to exact coordinate minimization than the projected gradient (PG) method. This is because AltMin performs exact minimization along each coordinate direction, leading to more efficient progress per iteration compared to the fixed step updates in PG. The faster run time for the PG method can likely be attributed to problem complexity or operation optimizations in the instructor's code. Overall, the AltMin function appears to be more efficient, though slightly slower.

Problem 2

In the lecture, we derived the proximal gradient method for solving the Lasso problem

$$\underset{\mathbf{x} \in \mathbb{R}^n}{\text{minimize}} \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|^2 + \lambda \|\mathbf{x}\|_1, \quad (\text{Lasso})$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a sensing matrix, $\mathbf{b}$ is the measurement vector, and $\lambda > 0$ is the regularization parameter.

1. Write a solver for (Lasso) by the proximal gradient method. Use the instructor's provided file `PG_Lasso.m` to write a Matlab function `PG_Lasso` with input $\mathbf{A}$, $\mathbf{b}$, initial vector $\mathbf{x}_0$, parameter `lam`, and tolerance `tol`, and with a provided stopping condition.

Proximal Gradient Method for Solving Lasso

```
function [x,hist_res] = PG_Lasso(A,b,x0,lam,tol)

% proximal gradient method for solving the Lasso
% min_x .5*||A*x-b||^2 + lam*||x||_1

% compute Lipschitz constant
L = norm(A)^2;

% compute gradient at x0
grad = A' *(A * x0 - b);

% perform one proximal gradient step with stepsize 1/L to get a new x

x = sign(x0 - (1/L)*grad) .* max(abs(x0 - (1/L)*grad) - lam/L, 0);

% evaluate norm of the proximal gradient mapping
res = L * norm(x-x0);

hist_res = res;

while res > tol
    x0 = x;
```

```
% compute gradient at x0
grad = A' * (A * x0 - b);

% perform one proximal gradient step with stepsize 1/L to get a new x

x = sign(x0 - (1 / L) * grad) .* max(abs(x0 - (1 / L)*grad) - lam/L, 0);

% evaluate norm of the proximal gradient mapping
res = L * norm(x-x0);
hist_res = [hist_res; res];
end

end
```

-
2. Write a solver for (Lasso) by the accelerated proximal gradient method. Use the instructor's provided file `APG_Lasso.m` to write a Matlab function `APG_Lasso` with input **A**, **b**, initial vector **x0**, parameter **lam**, and tolerance **tol**, and with a provided stopping condition.

Accelerated Proximal Gradient Method for Solving Lasso

```
function [x,hist_res] = APG_Lasso(A,b,x0,lam,tol)

% accelerated proximal gradient method for solving the Lasso
% min_x .5*||A*x-b||^2 + lam*||x||_1

% compute Lipschitz constant
L = norm(A)^2;

% compute gradient at x0
grad = A' * (A * x0 - b);

% perform one proximal gradient step with stepsize 1/L to get a new x
x = sign(x0 - (1/L)*grad) .* max(abs(x0 - (1/L)*grad) - lam/L, 0);

% evaluate norm of the proximal gradient mapping
res = L * norm(x-x0);

hist_res = res;

x0 = x;

% y used to denote the extrapolated point

y = x;

% used to compute the extrapolation weight
t0 = 1;

while res > tol
```

```
% compute a gradient at y
grad = A' * (A * y - b);

% perform one proximal gradient step with stepsize 1/L to get a new x
x = sign(y - (1/L)*grad) .* max(abs(y - (1/L)*grad) - lam/L, 0);

% update t value to compute extrapolation weight

t1 = (1 + sqrt(1 + 4 * t0^2)) / 2;

% update the extrapolated point y

y = x + ((t0 - 1) / t1) * (x - x0);

% evaluate norm of the proximal gradient mapping
res = L * norm(x-x0);
hist_res = [hist_res; res];

t0 = t1; x0 = x;
end

end
```

-
3. Test your two solvers by running the provided test file `test_Lasso.m` and compare to the instructor's function. Print your code and the results you get. What do you observe on the convergence speed of the two methods?

Code Results: **test_lasso.m Output**

```
>> test_Lasso
Results by student solver
PG method: time = 0.1512, Relative Error = 1.5733e-02
APG method: time = 0.0294, Relative Error = 1.4732e-02
Results by Instructor solver
PG method: time = 0.1485, Relative Error = 1.5733e-02
APG method: time = 0.0391, Relative Error = 1.4732e-02
```

Student Figure Result

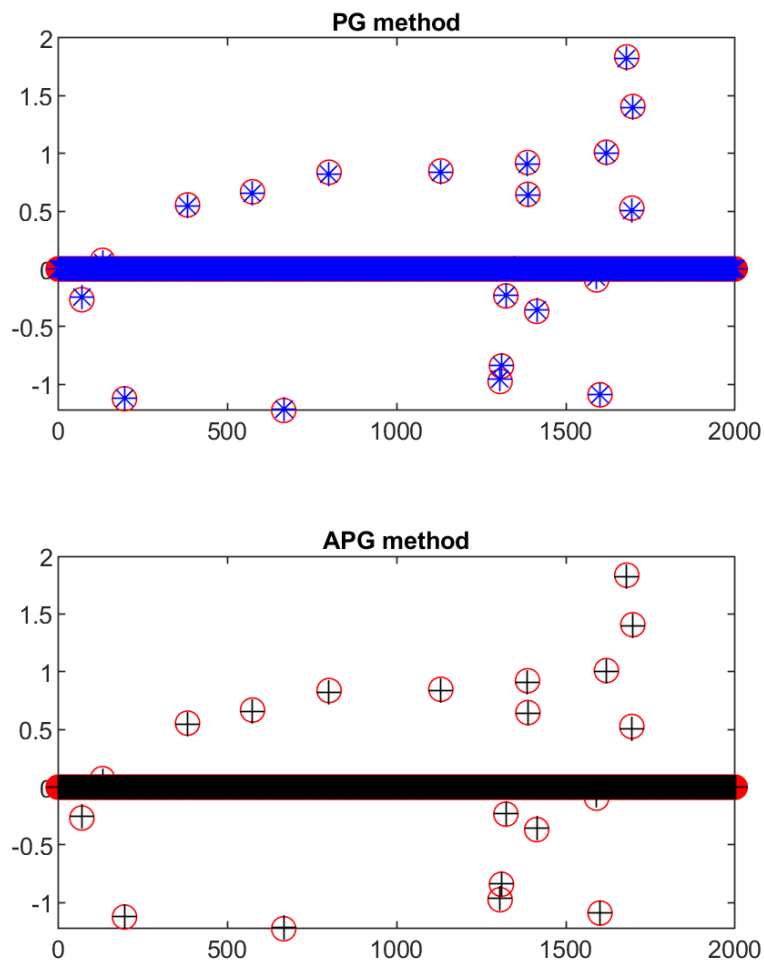


Figure 6: Student test_Lasso Result

Instructor Figure Result

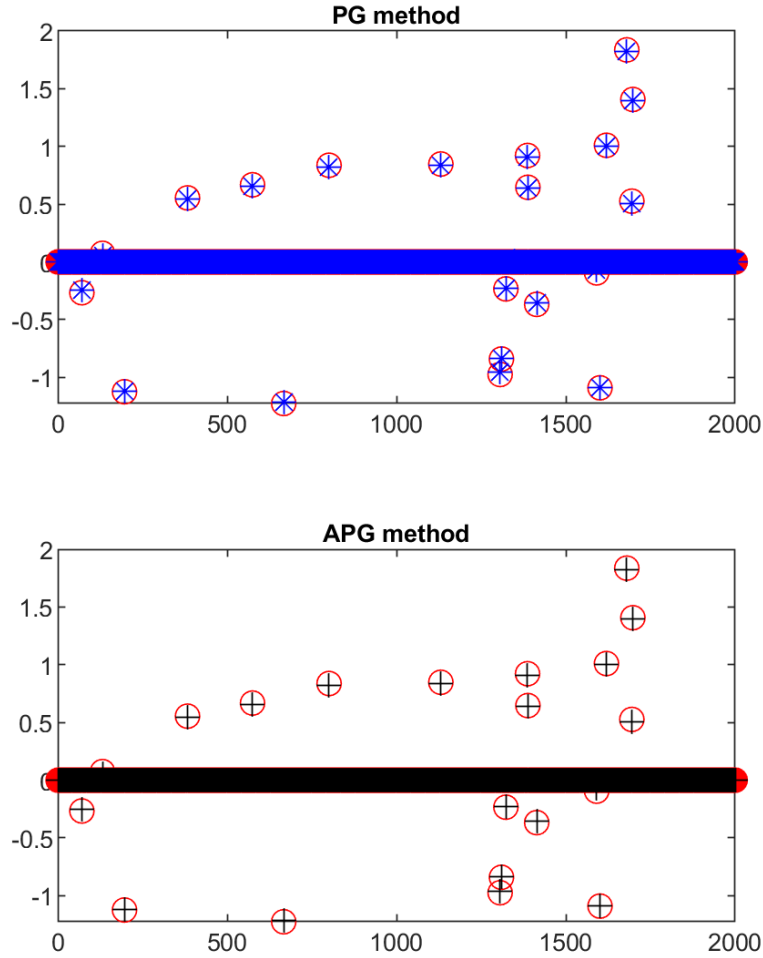


Figure 7: Instructor test_Lasso Result

We have nearly identical performances of both the PG and APG functions, so we conclude our functions are working correctly.

Now comparing the speed of the two methods. Based on the outputs, the accelerated proximal gradient (APG) method converges to a lower residual faster than the standard proximal gradient (PG) method in terms of iterations and runtime. This is because APG uses a momentum-based extrapolation that accelerates convergence, while PG uses fixed step updates that progress more conservatively.